

Code-Layout, Readability and Reusability

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples (Files/Folders)
1	Folder/Directory Structure	<i>The project follows a clear Flask directory structure by separating frontend files, backend logic, and static assets into dedicated folders.</i>	<i>templates/, static/css/, static/js/, app.py</i>
2	Naming Conventions	<i>Python functions use snake_case, variables have meaningful names, and HTML/CSS/JavaScript files use descriptive lowercase names for clarity and consistency.</i>	<i>generate_hook(), generate_script(), style.css, script.js</i>
3	Code Readability & Comments	<i>The source code is divided into logical sections with descriptive comments explaining application setup, helper functions, API routes, and configuration.</i>	<i>app.py section headers and inline comments</i>

4	Modularity & Reusability	<i>Each AI feature is implemented as an independent helper function (generate_hook(), generate_script(), generate_scenes(), generate_cta()), allowing code reuse across multiple API endpoints and reducing duplication.</i>	<i>app.py helper functions and /api/generate-all endpoint</i>
----------	-------------------------------------	--	---

Date	15 March 2026
Team ID	SWUID2026234324
Project Name	VideoScriptAI – AI Powered Video Content Generator
Maximum Marks	5 Marks

Code Quality & Structure Details

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples (Files/Folders)
			<i>across 5 pages]</i>
5	Formatting & Linting Tools	<i>[List the tools used to enforce consistent styling automatically]</i>	<i>[e.g., Prettier and ESLint configured in package.json]</i>
6	Error Handling Patterns	<i>[Describe the consistent approach to catching and returning errors]</i>	<i>[e.g., Centralized error middleware in Express server.js]</i>